Complete the following programming exercises on Classes, Objects, Access Control, Inheritance, Polymorphism, Abstraction, and Encapsulation. Task 1: Basic Class Hierarchy (Inheritance)

- Define a base class Vehicle with brand and fuel_type as instance attributes (use **init**).
- Create a subclass Car that inherits from Vehicle and adds num_doors as an additional attribute.
- Ensure the Car class calls the parent class's **init** method (using super() in Python).

Task 2: Polymorphism with Methods

- Define an abstract Shape class with an abstract method area() (use ABC and abstractmethod in Python).
- Create Circle and Rectangle subclasses that inherit from Shape.
- Implement area() in each subclass (for Circle, use πr^2; for Rectangle, use length * width).

Task 3: Encapsulation with Access Control

- Define BankAccount with a private attribute __balance (use double underscore in Python).
- Provide public methods:
  - deposit(amount) to add to __balance.
  - withdraw(amount) to deduct from __balance (check for sufficient funds).
- Use getter methods if balance access is needed (e.g., get_balance()).

Task 4: Abstraction with Base Class

- Create an abstract Animal class with an abstract method make_sound().
- Implement subclasses Dog and Cat, each overriding make_sound() to return "Woof!" and "Meow!" respectively.
- Use ABC and abstractmethod decorators in Python.

Task 5: Constructor and Destructor

- Define a Person class with **init**(self, name) to set the name attribute.
- Add a destructor **del**(self) that prints a farewell message (e.g., "Goodbye, {name}!").
- Test by creating and deleting an instance (use del explicitly or let it go out of scope).

Confirmed the abstract classes correctly block direct instantiation, and everything else runs and prints as expected. Here's the full solution, pasted in rather than as a separate file, with a short explanation of what each part demonstrates.

Task 1: Inheritance

```
class Vehicle:
```

```
    def __init__(self, brand, fuel_type):
        self.brand = brand
        self.fuel_type = fuel_type


class Car(Vehicle):
    def __init__(self, brand, fuel_type, num_doors):
        super().__init__(brand, fuel_type)
        self.num_doors = num_doors
```

`Car` extends `Vehicle` and calls `super().__init__()` so it does not have to repeat the logic for setting `brand` and `fuel_type`. This is the standard way to build a hierarchy where a subclass reuses and extends a parent's behaviour rather than duplicating it.

Task 2: Polymorphism

```
from abc import ABC, abstractmethod
import math


class Shape(ABC):
    @abstractmethod
    def area(self):
        pass


class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return math.pi * self.radius ** 2


class Rectangle(Shape):
    def __init__(self, length, width):
        self.length = length
        self.width = width

    def area(self):
        return self.length * self.width
```

Both subclasses implement `area()` in their own way, so calling `shape.area()` in a loop over a mixed list of shapes produces the correct result for each type without needing to check what type of object it is first. That is the essence of polymorphism.

Task 3: Encapsulation

```
class BankAccount:
    def __init__(self, balance=0):
        self.__balance = balance

    def deposit(self, amount):
        if amount <= 0:
            print("Deposit amount must be positive.")
            return
        self.__balance += amount
```

```
    def withdraw(self, amount):
        if amount <= 0:
            print("Withdrawal amount must be positive.")
            return
        if amount > self.__balance:
            print("Insufficient funds.")
            return
        self.__balance -= amount

    def get_balance(self):
        return self.__balance
```

The double underscore triggers Python's name mangling, so `account.__balance` cannot be accessed directly from outside the class, which is what I confirmed when testing it (it raises an `AttributeError`). The only way to read or change the balance is through the public methods, which is the point of encapsulation.

Task 4: Abstraction

```
class Animal(ABC):
    @abstractmethod
    def make_sound(self):
        pass


class Dog(Animal):
    def make_sound(self):
        return "Woof!"


class Cat(Animal):
    def make_sound(self):
        return "Meow!"
```

`Animal` cannot be instantiated on its own since it has an unimplemented abstract method. It only exists to define a contract that `Dog` and `Cat` must fulfil, which I verified by attempting `Animal()` directly and getting a `TypeError`.

Task 5: Constructor and Destructor

```
class Person:
    def __init__(self, name):
        self.name = name

    def __del__(self):
        print(f"Goodbye, {self.name}!")


p = Person("Alex")
print(f"Created person: {p.name}")
del p
```

Running this prints "Created person: Alex" followed by "Goodbye, Alex!" once `del p` triggers the destructor and Python's garbage collector reclaims the object.

I tested the whole set together and every task behaves as expected, including the edge cases (withdrawing more than the balance, and trying to instantiate the abstract classes directly). Happy to package this as a `.py` file if you'd rather submit it that way.